# ExcelDex

**Personal Project 05**

**Jun Xiang**

**14 November 2026**

# Table of Contents

1. Overview ..... 3
1.1 Problem ..... 3
1.2 Solution ..... 3
1.3 Components ..... 3
1.4 Features ..... **Error! Bookmark not defined.**
2. Scope ..... 4
2.1 In Scope ..... 4
2.2 Out of Scope ..... 4
3. Setup & Design ..... 4
4. Methodology ..... 5
5. Limitations & Risks ..... 6
6. Testings and Findings ..... 6
7. Challenges ..... 6
8. Improvements ..... 7

# 1. Overview

ExcelDex is a search-first quick reference for Excel functions and tips. It runs in the browser with no backend.

Users search by function name or by describing what they want to do, and get a plain-English explanation, sample data and a formula they can edit and copy into Excel.

## 1.1 Problem

Finding the right Excel function or shortcut often means knowing its exact name or reading long tutorials. This makes simple questions, such as how to look up a value or combine text, slower to answer than they should be.

## 1.2 Solution

ExcelDex answers "how do I do this in Excel?" with one search. Results appear as the user types, and each answer explains the function in plain language with a worked sample and an editable formula.

## 1.3 Goals

- Answer a task question with one search
- Explain functions in plain language, with sample data that makes ranges like A2:A5 meaningful
- Be clear about sources, and flag shortcuts that are not in Microsoft's documentation
- Measure search quality instead of assuming it works
- Stay small, static, accessible and well tested

## 1.4 Features

| Feature | Description |
|---|---|
| Search | Find entries by name, task, typo, word form, or text inside steps and notes. |
| Formula Builder | Edit the arguments of a formula, add optional arguments, reset, and copy. |
| Live Explanation | The plain-English explanation rewrites itself as the formula changes. |
| Sample Data | Every example shows its sample data and the result. |
| Tips | Keyboard shortcuts and how-tos, with Windows and Mac keycaps marked "hold together" or "press in turn". |

| | |
|---|---|
| Filters | Home page filters for Functions, Tips and each topic, with multiple selections allowed. |
| Themes & Layout | Light and dark themes, keyboard-friendly search, and a responsive layout. |
| Privacy | No accounts, no analytics, no third-party requests, and self-hosted fonts. |

# 2. Scope

## 2.1 In Scope

- Function entries and tips grouped by topic.
- Search by name, partial name, task, typo, word form, and body text.
- A formula builder with required and optional arguments.
- Plain-English explanations, sample data and results for every example.
- Source links and last-checked dates for each entry.
- Light and dark themes with a responsive layout.
- Automated accessibility, privacy and search-quality checks.

## 2.2 Out of Scope

The current implementation does not provide:

- A backend, database or user accounts.
- An AI assistant.
- A spreadsheet engine that calculates real workbooks.
- A learning course or teaching platform.

# 3. Setup & Design

## 3.1 Technology Stack

ExcelDex is a client-side web application built with:

- React and TypeScript for the interface and logic.
- CSS for styling, design tokens and responsive layout.
- Vite for development and building.
- JSON for the content (functions and tips).
- Vitest, Testing Library and axe-core for automated tests.
- Linting with oxlint.

### 3.2 System Architecture

ExcelDex separates the content, search, formula handling and interface into different parts of the project.

- The data module holds the entries in excelEntries.json and typed access to them.
- The search module normalises queries, stems words, scores entries and tolerates typos.
- The formula module parses a formula, rebuilds it, and explains it in words.
- The components contain the search box, home page, answer panel and keycaps.
- The eval folder holds the search evaluation queries, metrics and report.

### 3.3 Design Decisions

- The site is static with no backend, so it can be hosted anywhere and has nothing to keep running.
- The formula builder only lets users edit arguments, so the formula cannot be broken by accident.
- Every entry links to the Microsoft page it was checked against, with the date checked.
- Shortcuts missing from Microsoft's documentation are kept but carry a visible note.
- There are no third-party requests, and a test fails if any are added.

### 3.4 Running the Project

Install the project dependencies with npm install, then run npm run dev to start the development server.

## 4. Methodology

ExcelDex was built in small steps, developing the content, search and interface together instead of writing many entries before checking search. Each query is cleaned, split into stemmed words and scored against every entry, with adjustments for typos, rare words and how many of the query's words an entry matches.

Search was measured with an evaluation set of test queries, split into tuning and held-out sets, and compared against the original simple matcher. The project also runs type checking, linting and a production build.

## 5. Limitations & Risks

- Search has no synonym list. If a word never appears in an entry, the entry cannot be found by it.
- Search cannot say "nothing relevant", so a query such as "pivot table" still returns weak matches.
- Gains on the tuning queries did not fully carry over to the held-out queries.
- All queries were written by the author, so they show how search behaves, not how strangers would use it.
- Mac shortcuts come from Microsoft's list and have not been tested on a Mac.
- Accessibility has not been checked with real screen readers, Safari, Firefox or real phones.

## 6. Testings and Findings

ExcelDex includes automated tests covering search, formula parsing, entry data, clipboard and keyboard behaviour, accessibility, colour contrast and privacy.

- Run npm test to run the automated test suite.
- Run npm run build to type-check and build for production, and npm run eval to produce the search evaluation report.

On the held-out queries, typo tolerance, stemming and reading body text gave the biggest gains over the original matcher. The tests catch regressions, but they do not replace testing with real users, screen readers or other browsers.

## 7. Challenges

| Challenge | How it was handled |
|---|---|
| Finding answers without knowing the name | Search scores task phrases, keywords, descriptions and body text, with typo tolerance and stemming. |
| Measuring search quality honestly | Queries are split into tuning and held-out sets, and the report shows results with and without word-for-word copies. |
| Explaining formulas that change | The formula is parsed and rebuilt from its arguments, and the explanation is regenerated each time. |

| | |
|---|---|
| Staying accessible and private | axe-core and contrast tests run on the real screens, and a test fails on any third-party request. |

## 8. Improvements

- **Search Quality**
  - Collect real queries and add them to the evaluation file.
  - Label weak matches as closest results instead of presenting them as answers.
- **Content**
  - Split the Math topic and add more tips.
  - Add synonyms through shared concept IDs.
- **Language**
  - Support English and Chinese search.
- **Accessibility & Compatibility**
  - Test with real screen readers, Safari, Firefox and real phones.